

Assignment: APIs

Description

For your project, implement the following:

- **Backend:** A RESTful API that returns JSON, ready to be consumed by the frontend.
- **Frontend:** Asynchronous JavaScript that fetches data from the backend, uses Promises to process the response, and renders the result in the DOM.

Grading criteria

The student can demonstrate and explain the following:

1. What an asynchronous request is, and what role Promises play in handling one.
2. How the framework handles different response formats (JSON vs. HTML).
3. How an asynchronous request is made in JavaScript.
4. How an asynchronous response is processed in JavaScript.

Grading rubric

Criterion	Insufficient	Approaching	Sufficient
1. Async & Promises	Cannot explain what an async request is, or confuses it with a synchronous request. The role of Promises is unclear or missing.	Can describe async requests in general terms but struggles to explain how Promises fit in (e.g., cannot explain <code>.then()</code> , <code>async/await</code> , or error handling).	Clearly explains what an async request is and why it is non-blocking. Accurately describes the role of Promises, including how the resolved value is used and how errors are caught.
2. Format handling in the framework	Cannot explain how the framework returns JSON. Does not see the difference between an HTML and a JSON response.	Knows that JSON and HTML responses exist and can point to the relevant code, but cannot explain <i>why</i> they differ or how the framework decides which to return.	Explains how the framework distinguishes between HTML and JSON responses (e.g., separate routes/controllers, <code>Content-Type</code> header). Can trace the response from the controller to the client.
3. Making an async request in JavaScript	No async request is present in the frontend code, or the code does not run.	An async request exists and reaches the backend, but contains errors or relies on copy-pasted code the student cannot explain.	The async request is correctly implemented and the student can explain each part: the URL, the method, any headers, and how the fetch is triggered.
4. Handling an async	The response is not processed, or the	The response is partially processed (e.g., data is	The response is fully processed: the JSON is

Criterion	Insufficient	Approaching	Sufficient
response in JavaScript	data never reaches the DOM.	logged to the console but not rendered, or only a hardcoded value is shown in the DOM).	parsed, the relevant data is extracted, and the result is correctly rendered in the DOM. Error cases (e.g., failed fetch) are handled.